UNIVERSITY of SAN CARLOS

SCIENTIA • VIRTUS • DEVOTIO

SYNTAX & SEMANTICS

Programming Languages

Syntax vs. Semantics

Syntax

- Structure or form of statements.

Ex. int num = 5;

Semantics

- Meaning or behavior of statements.

Ex. Integer 5 assigned to variable num

Relationship of Syntax & Semantics

Good language design:

- Meaning follows appearance
- Syntax suggests behavior

Syntax is easier to describe than semantics.

Sentences

Strings of a language, also called ***statements***.

Lexemes – small units of a language

- Includes numeric literals, operators, and special words, among others
- Forms group called ***identifiers.***
- A ***token*** of a language is a category of its lexemes.

```
index = 2 * count + 17;
```

Lexemes	*Tokens*
`index`	identifier
`=`	equal_sign
`2`	int_literal
`*`	mult_op
`count`	identifier
`+`	plus_op
`17`	int_literal
`;`	semicolon

Language Recognizers

A recognizer:

- Determines if a string belongs to a language.
- Accepts or rejects input.

Ex. Compiler syntax analyzer (parser)

Language Generators

A generators:

- Produces valid sentences of a language.

Ex. Compiler syntax analyzer (parser)

UNIVERSITY of SAN CARLOS

SCIENTIA • VIRTUS • DEVOTIO

Formal Methods of Describing Syntax

Context-free Grammars

is the formal mathematical system used to define the syntax of languages.

G = (V, E, R, S)

- **Terminals (E)**The basic symbols from which strings are formed. These are your **Tokens** (e.g., if, +, id, numbers).
- **Variables / Non-terminals (V):** Placeholders that stand for patterns of strings (e.g., <sentence>, <expression>)
- **Start Symbol (S):** The special variable where the derivation begins (usually the top-level concept, like <Program>).
- **Productions / Rules (R):** The rules that say how to replace Variables with Terminals or other Variables.

Backus-Naur Form

Component	Symbol	Description
Non-Terminal	<name>	A placeholder or category (like <sentence> or <loop>). These need to be broken down further.
Terminal	"text"	The final pieces (like "if", "+", or 0-9). **These are your Tokens.** They cannot be broken down further.
Definition	::=	Means "is defined as" or "can be replaced by".
Choice	`	`

A Simple Example (An Integer)

<integer> ::= <sign> <digits>
<sign> ::= "+" | "-" | <empty>
<digits> ::= <digit> | <digit> <digits>
<digit> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

How to read this:

1. **`<integer>`**: An integer consists of a `<sign>` followed by `<digits>`.
2. **`<sign>`**: A sign can be a plus, a minus, or nothing (empty).
3. **`<digits>`**: This is a **recursive** rule. It says digits are either a single `<digit>`, OR a single `<digit>` followed by more `<digits>`. This allows us to have numbers of infinite length (e.g., `1`, `10`, `199`, `1000...`).
4. **`<digit>`**: These are the actual terminals ("0" through "9").

Metalanguage

a language that is used to describe another language.

BNF is a metalanguage for programming languages.

Ex. <assign> -> <var> = <expression>

Grammars and Derivations

- Sentences of the languages starts with a special nonterminal of the grammar called the **start symbol**.

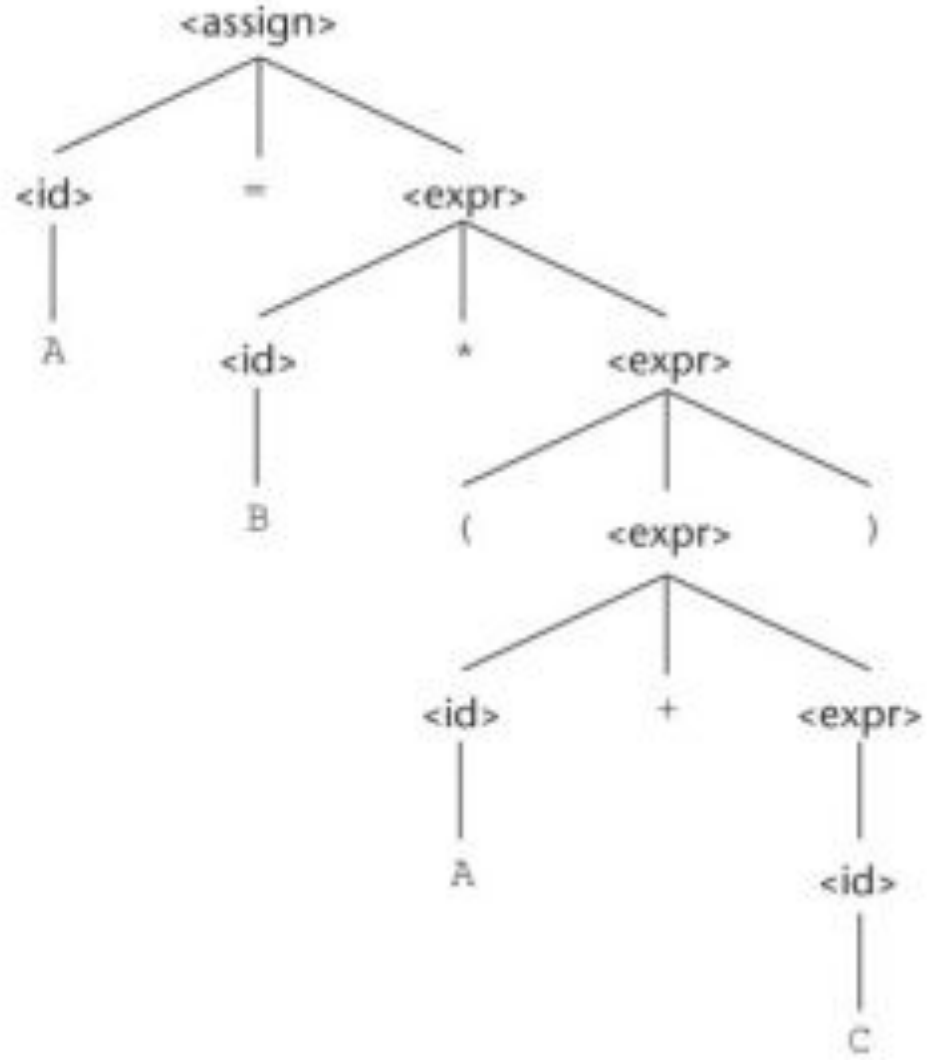
- Sequence of rule applications is called a **derivation**.

Grammars and Derivations

- <program> → **begin** <stmt_list> **end**
- <stmt_list> → <stmt>
- | <stmt> ; <stmt_list>
- <stmt> → <var> = <expression>
- <var> → `A` | `B` | `C`
- <expression> → <var> + <var>
- | <var> - <var>
- | <var>

Parse Trees

hierarchical syntactic structure of the sentences of the languages.



UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO